

o_que_e_stork

O que é pacote de pesquisa de site stork

- OBJETIVO
- PASSO 01 - Instalação do stork no sistema operacional linux
- PASSO 02 - **Configurando o seu site para hospeda o stork:**
- PASSO 03 - Como criar um índice de pesquisa para o stork
- PASSO 04 - Como criar uma página de pesquisa usando o pacote stork instalado em <https://files.stork-search.net>
- PASSO 05 - Como criar uma página de pesquisa usando o pacote stork instalado no seu site
- Testando o índice criado por stork
- REFERÊNCIAS

OBJETIVO

1. O pacote de pesquisa [stork](#) tem como objetivo indexar um [site estático](#) para fazer pesquisas no mesmo usando apenas as linguagens javascript e webassembly.
2. Motivo:
 1. O servidor web [github.io](#) não permite instalar programas *cgi*, *fastcgi* ou *módulo apache* no *servidor web* sem que pague mensalidade para o mesmo.

PASSO 01 - Instalação do stork no sistema operacional linux

1. Usando o [gerenciador de pacotes rust](#)
 1. Instalar [Rust toolchain](#) para que o pacote cargo fique disponível

```
curl https://sh.rustup.rs -sSf | sh

Rust is installed now. Great!
```

2. Instalar o pacote Stock

```
cargo install stork-search --locked
```

3. Referência

1. [Installing Stork...](#)

PASSO 02 - **Configurando o seu site para hospeda o stork:**

1. Baixe [stork.js](#), [stork.wasm](#) e [stork.js.map](#) da [Versão Stork - v1.6.0 Latest](#) do [Github](#) e publique esses arquivos em seu *servidor web*.
2. Certifique-se de servir *stork.wasm* com o tipo *MIME* correto e certifique-se de chamar *stork.initialize("stork.wasm")* com o *URL onde você está hospedando* o arquivo *stork.wasm*.
3. Sirva esses arquivos de seu *servidor web*. Carregue os arquivos Javascript *stork.js* e *WASM stork.wasm* em seu servidor ou host da Web e os *apresente publicamente* como faria com uma folha de *estilo* ou *imagem*.
4. Certifique-se de servir o arquivo *WASM* com o *application/wasm* tipo [MIME](#) . Se você estiver colocando esses arquivos atrás do seu [CDN](#), certifique-se de encaminhar o *Content-Type* cabeçalho da fonte do arquivo, por meio desse [CDN](#).
5. Inicialize o *Stork* em sua página da Web com o *URL do WASM*. A página Javascript avançado descreve o *stork.initialize()* método, que instrui o *Stork a carregar o WASM*. Certifique-se de chamar este método - mesmo se estiver usando a *stork.register()* API mais simples - *antes de chamar qualquer outro método Stork*:

```
<script src="/js/stork.js"></script>

<script>
  stork.initialize("/js/stork.wasm")
  stork.register("yourIndexName", "/index-file.st")
</script>
```

6. NOTAS

1. No site [pssp.app.br](#) o arquivo [stork.js](#) e [stork.wasm](#) deve ficar na pasta */js*.
2. [Usando uma API JavaScript do WebAssembly](#).
3. Como escrever políticas de segurança de conteúdo:
 1. Stork conta com a capacidade de executar *WASM* no navegador. A maioria das definições de política de segurança de conteúdo bloqueará a execução do *WASM* por padrão. Você pode ver este manifesto como um erro de carregamento *WASM* genérico:

```
Uncaught (in promise) StorkError: Error while loading
WASM at /stork.wasm
```

```
Wasm code generation disallowed by embedder
```

2. Infelizmente, não há muitas boas opções para ter um [CSP](#) restritivo e executar o *WASM* no navegador, mesmo que você *auto-hospede* o arquivo *WASM*. Para que o *Stork* funcione, você terá que *permitir a execução do WASM dentro de sua script-src diretiva*. O seguinte [CSP](#) é o [CSP](#)

minimamente permissivo que ainda permitirá que o *Stork* funcione em todos os principais navegadores:

```
<meta
  http-equiv="Content-Security-Policy"
  content="default-src 'self'; script-src 'unsafe-
eval';"
/>
```

3. O Google Chrome oferece suporte às políticas '*wasml-unsafe-eval*'(ou, às vezes,) em sua diretiva para permitir a execução do *WASM*. No entanto, no momento em que este livro foi escrito, o *Safari* e o *Firefox* exigem que você especifique a política em sua diretiva, o que torna seu *CSP* restritivo muito menos restritivo.'*wasml-eval*'*script-srcunsafe-evalscript-src*.

7. REFERÊNCIAS

1. [Download Stork - v1.6.0 Latest](#)

PASSO 03 - Como criar um índice de pesquisa para o stork

1. Para criar um índice para arquivo [stork.js](#), é necessário instalar o pacote *stork* descrito no [passo 01](#) acima.
2. Cada arquivo da lista de arquivos as serem pesquisados, devem ter as palavras `_ \ _ \` para marcar o trecho de código que deve ser pesquisado.

1. Exemplo:

```

<!-- markdownlint-disable-next-line -->
<link type="text/css" href="../../css/defaulttheme.css"
rel="stylesheet" />

<!-- markdownlint-disable-next-line -->

# Reflexão sobre fake, fatos e opinião

> 1. Qual a diferença entre: fake x fatos x opinião?
> 1. Fake news são textos que não descrevem a verdade
ou descrevem verdades fora do contexto.
> 1. Exemplo de mentira:
> 1. O Brasil foi descoberto por Portugal.
(Se havia morador ele não estava perdido).
> 2. Exemplo de verdade fora do contexto:
> 1. Uma doença enraizada em anos anteriores,
pode ser anunciada no momento atual com imagens da época em que
ela foi enraizada.
>
> 2. Fatos são textos que descrevem a verdade:
> 1. Portugal não conhecia o Brasil até o ano de 1500.
> 3. Opinião são textos que descrevem pontos de vista
pessoal ou cultural:
> 1. Portugal ao chegar no Brasil e perceber que as
pessoas que aqui vivia não possuía a pólvora, decidiram se
apropriar.
> 2. Os políticos do Brasil são desonestos.

<!-- markdownlint-disable-next-line -->

```

3. O programa de linha de comando *stork*, receberá um arquivo de configuração e criará um *arquivo de índice*. Um arquivo de índice é um *blob serializado e compactado* que a biblioteca *Stork Javascript* baixará e analisará. Geralmente tem a extensão de arquivo *.st* e tem aproximadamente o tamanho de um *JPEG*.
4. Para criar um índice de pesquisa, o projeto *Stork* requer que se crie um *arquivo de configuração* que declare quais documentos devem ser indexados. Este arquivo de configuração deve ter o formato de arquivo [TOML](#).
 1. Arquivo exemplo de meta-dados que o stork espera:

```
# index_stork.toml
[input]
base_directory = ""
url_prefix = ""
files = [
  {path =
"./linguas_humanas/portugues/producao_de_textos/fake_fato_opinioao
url = "0001", title = "Reflexão sobre fake, fatos e opinião"},
  {path =
"./linguas_humanas/portugues/producao_de_textos/descricao_de_texto
url = "0002", title = "Estudo da língua portuguesa"},
  {path = "./programacao/html/js/stork/o_que_e_stork.html",
"0001", title = "O que é pacote de pesquisa de site stork"}
]
```

1. Notas:

1. *base_directory* = diretório base que deve ser usado para resolver caminhos relativos. Esse caminho estará relacionado ao diretório de trabalho quando você executar o comando *stork build*.
 2. *url_prefix* = ???Não compreendi do que se trata
 3. *files* = A lista de arquivos e suas pastas juntamente com seus títulos
 4. [Veja todos os atributos aceitos pelo arquivo *basic.toml* ...](#)
 5. [Veja como o arquivo *basic.toml* pode ter outros formatos de declaração...](#)
2. Exemplo de como criar o índice *list_stork.st* usando o arquivo *index_stork.toml* descrito acima:

```
# Cria índice para stork.js
stork build --input index_stork.toml --output list_stork.st
```

1. Notas

1. Todos os caminhos de arquivo no arquivo de configuração, incluindo o diretório base, são relativos ao diretório de trabalho quando você executa o comando Stork . Por exemplo, se você estiver no diretório *~/project*, ao executar *_stork build --input index_stork.toml --output list_stork.st*, o Stork procurará os arquivos em:
 1. *./project/my_files/federalist-1.txt*
 2. *./project/my_files/federalist-2.txt*
 3. *./project/my_files/federalist-3.txt*

5. Referências:

1. [Indexação de arquivos HTML e Markdown](#)
2. [Indexação de conteúdo não inglês](#)

PASSO 04 - Como criar uma página de pesquisa usando o pacote stork instalado em <https://files.stork-search.net>

1. Use o modelo abaixo para criar sua página de pesquisa:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <title>Federalist Search</title>

    <!-- O Stork é altamente personalizável e, se você quiser,
pode escrever CSS que
        faça com que a saída do Stork pareça exatamente como
você deseja. Como alternativa,
        você pode usar um dos temas pré-existentes para criar
rapidamente uma aparência polida
        para sua interface de pesquisa. Aqui, estamos usando o
tema Básico: -->
    <link
      rel="stylesheet"
      href="https://files.stork-
search.net/releases/v1.6.0/basic.css"
    />

  </head>

  <body>
    <!-- Cada interface de pesquisa deve ter um elemento de
entrada e um elemento de saída .
        (O wrapper é usado apenas pelo tema basic.css que
incluimos acima.) -->
    <div class="stork-wrapper">
      <input data-stork="federalist" class="stork-input" />
      <div data-stork="federalist-output" class="stork-
output"></div>
    </div>

    <!-- Você pode carregar a biblioteca Javascript do Stork
CDN (com suporte do AWS Cloudfront) ou auto-hospedá-la.
        Logo antes da </body>tag de fechamento, inclua o script
Stork e registre o index.-->
    <script src="https://files.stork-
search.net/releases/v1.6.0/stork.js"></script>
    <script>
      stork.register(
        'federalist',
        'https://files.stork-
search.net/releases/v1.6.0/federalist.st'
      )
    </script>
  </body>

</html>
```

2. Nota:

1. Onde tiver a palavra *federalist* troca o nome do seu índice criado em...
2. [Neste exemplo de uso o índice está hospedado em...:](#)
3. [O arquivo css de entrada e saída está hospedado em...:](#)
4. [O programa Stork está hospedado em...:](#)
5. O Código fonte foi do [projeto stork.js](#) foi script em *typescript*, e o arquivo stork.js foi gerado pelo transpilador.
6. [Construindo uma interface de pesquisa](#)

PASSO 05 - Como criar uma página de pesquisa usando o pacote stork instalado no seu site

1. Use o modelo abaixo para criar sua página de pesquisa e salve o arquivo na raiz do site:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <title>Titulo de sua pesquisa</title>

    <!-- O Stork é altamente personalizável e, se você quiser,
pode escrever CSS que
    faça com que a saída do Stork pareça exatamente como
você deseja. Como alternativa,
    você pode usar um dos temas pré-existentes para criar
rapidamente uma aparência polida
    para sua interface de pesquisa. Aqui, estamos usando o
tema stork.css: -->
    <link
      rel="stylesheet"
      href="/css/stork.css"
    />

  </head>

  <body>
    <!-- Cada interface de pesquisa deve ter um elemento de
entrada e um elemento de saída .
    (O wrapper é usado apenas pelo tema stork.css que
incluimos acima.) -->
    <div class="stork-wrapper">
      <input data-stork="list_stork" class="stork-input" />
      <!-- <div data-stork="list_stork" class="stork-output">
</div> -->
      <div data-stork="list_stork-output" class="stork-
output"></div>
    </div>

    <!-- Código javascript deve está localizado na pasta /js .
    O código /js/stork.js depende do código /js/stork.wasm
    -->
    <script src="/js/stork.js"></script>

    <!-- O código javascript precisa saber o nome da lista que
le deve pesquisar -->
    <script>
      stork.initialize("/js/stork.wasm")
      stork.register('list_stork', 'list_stork.st' )
    </script>

  </body>

</html>
```

2. Nota:

1. Onde tiver a palavra *federalist* troca o nome do seu índice criado em...
2. [Neste exemplo de uso o índice está hospedado em...:](#)
3. [O arquivo css de entrada e saída está hospedado em...:](#)
4. [O programa Stork está hospedado em...:](#)
5. O Código fonte foi do [projeto stork.js](#) foi script em *typescript*, e o arquivo stork.js foi gerado pelo transpilador.
6. [Construindo uma interface de pesquisa](#)

Testando o índice criado por stork

1. Depois de escrever um arquivo de configuração, você pode querer testar o funcionamento da interface de pesquisa antes de carregá-lo em seu site. O Stork oferece um modo de teste, no qual ele criará seu índice de pesquisa e o carregará em uma interface da Web simplificada para que você possa brincar com a funcionalidade de pesquisa enquanto itera em sua configuração.
2. Para testar seu índice, execute:

```
stork test --config basic.toml  
# or  
stork test --index my-index.st
```

1. Nota:

1. Após executar o comando acima executar no browser <http://localhost:1612>, a página web servida pela Stork.

REFERÊNCIAS

1. [Link da versão mais recente do Stork no Github](#)
2. [Construindo uma interface de pesquisa](#)
3. <https://github.com/jameslittle230/stork>
4. [Stork Demo 1](#)
5. [Stork Turns One: Construindo uma ferramenta de busca para sites estáticos com Rust e WebAssembly](#)
6. [Log de alterações do stork](#)
7. [Estou encerrando meu trabalho com Stork.](#)
8. [Stork: Pesquisa na Web incrivelmente rápida, com James Little](#)
9. [stork-and-netlify - pelican](#)

